

信息的表示和处理

现代计算机存储和处理的信息以二值信号表示。这些微不足道的二进制数字，或者称为位(bit)，形成了数字革命的基础。大家熟悉并使用了 1000 多年的十进制(以 10 为基数)起源于印度，在 12 世纪被阿拉伯数学家改进，并在 13 世纪被意大利数学家 Leonardo Pisano(大约公元 1170—1250，更为大家所熟知的名字是 Fibonacci)带到西方。对于有 10 个手指的人类来说，使用十进制表示法是很自然的事情，但是当构造存储和处理信息的机器时，二进制值工作得更好。二值信号能够很容易地被表示、存储和传输，例如，可以表示为穿孔卡片上有洞或无洞、导线上的高电压或低电压，或者顺时针或逆时针的磁场。对二值信号进行存储和执行计算的电子电路非常简单和可靠，制造商能够在一个单独的硅片上集成数百万甚至数十亿个这样的电路。

孤立地讲，单个的位不是非常有用。然而，当把位组合在一起，再加上某种解释(interpretation)，即赋予不同的可能位模式以含意，我们就能够表示任何有限集合的元素。比如，使用一个二进制数字系统，我们能够用位组来编码非负数。通过使用标准的字符码，我们能够对文档中的字母和符号进行编码。在本章中，我们将讨论这两种编码，以及负数表示和实数近似值的编码。

我们研究三种最重要的数字表示。无符号(unsigned)编码基于传统的二进制表示法，表示大于或者等于零的数字。补码(two's-complement)编码是表示有符号整数的最常见的方式，有符号整数就是可以为正或者为负的数字。浮点数(floating-point)编码是表示实数的科学记数法的以 2 为基数的版本。计算机用这些不同的表示方法实现算术运算，例如加法和乘法，类似于对应的整数和实数运算。

计算机的表示法是用有限数量的位来对一个数字编码，因此，当结果太大以至不能表示时，某些运算就会溢出(overflow)。溢出会导致某些令人吃惊的后果。例如，在今天的大多数计算机上(使用 32 位来表示数据类型 int)，计算表达式 $200 * 300 * 400 * 500$ 会得出结果 -884 901 888。这违背了整数运算的特性，计算一组正数的乘积不应产生一个负的结果。

另一方面，整数的计算机运算满足人们所熟知的真正整数运算的许多性质。例如，利用乘法的结合律和交换律，计算下面任何一个 C 表达式，都会得出结果 -884 901 888：

```
(500 * 400) * (300 * 200)
((500 * 400) * 300) * 200
((200 * 500) * 300) * 400
400 * (200 * (300 * 500))
```

计算机可能没有产生期望的结果，但是至少它是一致的！

浮点运算有完全不同的数学属性。虽然溢出会产生特殊的值 $+\infty$ ，但是一组正数的乘积总是正的。由于表示的精度有限，浮点运算是不可结合的。例如，在大多数机器上，C 表达式 $(3.14 + 1e20) - 1e20$ 求得的值会是 0.0，而 $3.14 + (1e20 - 1e20)$ 求得的值会是 3.14。整数运算和浮点数运算会有不同的数学属性是因为它们处理数字表示有限性的方式不同——整数的表示虽然只能编码一个相对较小的数值范围，但是这种表示是精确的；而浮

点数虽然可以编码一个较大的数值范围，但是这种表示只是近似的。

通过研究数字的实际表示，我们能够了解可以表示的值的范围和不同算术运算的属性。为了使编写的程序能在全部数值范围内正确工作，而且具有可以跨越不同机器、操作系统和编译器组合的可移植性，了解这种属性是非常重要的。后面我们会讲到，大量计算机的安全漏洞都是由于计算机算术运算的微妙细节引发的。在早期，当人们碰巧触发了程序漏洞，只会给人们带来一些不便，但是现在，有众多的黑客企图利用他们能找到的任何漏洞，不经过授权就进入他人的系统。这就要求程序员有更多的责任和义务，去了解他们的程序如何工作，以及如何被迫产生不良的行为。

计算机用几种不同的二进制表示形式来编码数值。随着第3章进入机器级编程，你需要熟悉这些表示方式。在本章中，我们描述这些编码，并且教你如何推出数字的表示。

通过直接操作数字的位级表示，我们得到了几种进行算术运算的方式。理解这些技术对于理解编译器产生的机器级代码是很重要的，编译器会试图优化算术表达式求值的性能。

我们对这部分内容的处理是基于一组核心的数学原理的。从编码的基本定义开始，然后得出一些属性，例如可表示的数字的范围、它们的位级表示以及算术运算的属性。我们相信从这样一个抽象的观点来分析这些内容，对你来说是很重要的，因为程序员需要对计算机运算与更为人熟悉的整数和实数运算之间的关系有清晰的理解。

旁注 怎样阅读本章

本章我们研究在计算机上如何表示数字和其他形式数据的基本属性，以及计算机对这些数据执行操作的属性。这就要求我们深入研究数学语言，编写公式和方程式，以及展示重要属性的推导。

为了帮助你阅读，这部分内容安排如下：首先给出以数学形式表示的属性，作为原理。然后，用例子和非形式化的讨论来解释这个原理。我们建议你反复阅读原理描述和它的示例与讨论，直到你对该属性的说明内容及其重要性有了牢固的直觉。对于更加复杂的属性，还会提供推导，其结构看上去将会像一个数学证明。虽然最终你应该尝试理解这些推导，但在第一次阅读时你可以跳过它们。

我们也鼓励你在阅读正文的过程中完成练习题，这会促使你主动学习，帮助你理论联系实际。有了这些例题和练习题作为背景知识，再返回推导，你将发现理解起来会容易许多。同时，请放心，掌握好高中代数知识的人都具备理解这些内容所需要的数学技能。

C++ 编程语言建立在C语言基础之上，它们使用完全相同的数字表示和运算。本章中关于C的所有内容对C++都有效。另一方面，Java语言创造了一套新的数字表示和运算标准。C标准的设计允许多种实现方式，而Java标准在数据的格式和编码上是非常精确具体的。本章中多处着重介绍了Java支持的表示和运算。

旁注 C编程语言的演变

前面提到过，C编程语言是贝尔实验室的Dennis Ritchie最早开发出来的，目的是和Unix操作系统一起使用（Unix也是贝尔实验室开发的）。在那个时候，大多数系统程序，例如操作系统，为了访问不同数据类型的低级表示，都必须大量地使用汇编代码。比如说，像malloc库函数提供的内存分配功能，用当时的其他高级语言是无法编写的。

Brian Kernighan和Dennis Ritchie的著作的第1版[60]记录了最初贝尔实验室的C语言版本。随着时间的推移，经过多个标准化组织的努力，C语言也在不断地演变。1989